Studio Nx

S1-13 템포 변경 — 함수 설명 합본 (백엔드 308~310, JS 311)

2026-10-21

목차

1. ai-notes/backend(fastAPI_Python)/S1-13 - 템포 변경/308_models_schemas_tempo.md
2. ai-notes/backend(fastAPI_Python)/S1-13 - 템포 변경/309_services_tempo.md
3. ai-notes/backend(fastAPI_Python)/S1-13 - 템포 변경/310_api_tempo_full.md
4. ai-notes/javascript/S1-13 - 템포 변경/311_sequencer_tempo.md

백엔드 308 — 템포 변경점 모델과 스키마 (`models/tempo_change.py`, `schemas/tempo.py`)

항목	내용
파일	`studionx/backend/app/models/tempo_change.py`, `studionx/backend/app/schemas/tempo.py`
작업	S1-13 템포 변경: 템포 변경점 표의 모델과 요청, 응답 모양 (PRD 02 FR-202a)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 307 오디오 회귀 테스트 실행 명령, 다음 309 템포 서비스

1. 하는 일

곡 중간의 템포 변경점 한 건을 담는 모델(`TempoChange`)과, API가 주고받는 모양(스키마)을 정의합니다. 표 `tempo_changes`는 DB에 처음부터 있었지만(DB.md §3.3, 2026-10-05), 이 표를 읽고 쓰는 모델과 API가 없어서 템포 변경점은 저장도 조회도 할 수 없었습니다.

```
tempo_changes 한 행 (곡 중간의 템포 변경점)
├ project_id  – 프로젝트 (삭제하면 DB가 함께 지움)
├ tick        – 변경 위치. 0보다 커야 함 (0 의 템포는 projects.bpm). 프로젝트 안에서 유일
├ bpm         – 20.00 ~ 300.00
└ ramp        – STEP = 이 위치에서 바로 바뀜, LINEAR = 직전 변경점부터 이 위치까지 서서히 바뀜
```

시작 템포(tick 0)는 `projects.bpm`에 있고, 이 표는 그 뒤의 변경점만 가집니다. 실제 시간(초)은 이 지도로 계산하며, 오디오 엔진(`audio/timing.js`의 `ticksToSecondsWithChanges`)이 읽는 모양이 `{ tick, bpm, ramp }`입니다.

2. 언제 쓰나

쓰는 곳	하는 일
`tempo_service` (309)	목록 읽기와 통째 교체에 모델과 스키마를 씁니다.
`full_project_service` (310)	`GET /projects/{id}/full` 응답의 `tempo_changes`를 `TempoChangeItem`으로 채웁니다.
시험 `test_tempo_routes.py`	요청 검증(422)과 응답 모양을 확인합니다.

3. 코드

3.1 모델

```
class TempoRamp(StrEnum):
    """템포가 바뀌는 방식."""

    STEP = "STEP"
    LINEAR = "LINEAR"
```

```
class TempoChange(Base):
    """템포 변경점. 프로젝트마다 여러 개를 가진다."""
```

```
    __tablename__ = "tempo_changes"

    tempo_change_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE")
    )
    tick: Mapped[int] = mapped_column(INTEGER(unsigned=True))
    bpm: Mapped[Decimal] = mapped_column(Numeric(6, 2))
    ramp: Mapped[TempoRamp] = mapped_column(
        Enum(TempoRamp, native_enum=True), server_default=text("'STEP'")
    )
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

3.2 스키마

```
class TempoChangeInput(BaseModel):
    """템포 줄이 보내는 변경점 한 건."""

    tick: TempoTick
    bpm: TempoBpm
    ramp: Literal["STEP", "LINEAR"] = "STEP"

    @field_validator("bpm")
    @classmethod
    def round_bpm(cls, value: float) -> float:
        """DB 칸(DECIMAL(6,2))에 맞춰 소수 둘째 자리까지만 쓴다."""
        return round(value, 2)
```

```
class TempoChangeItem(TempoChangeInput):
    """응답 변경점 한 건 — TempoChangeInput 에 번호를 더한 모양."""

    tempo_change_id: int
```

```
class TempoChangesSaveRequest(BaseModel):
    """PUT /projects/{id}/tempo-changes — 이 프로젝트의 템포 변경점을 보낸 목록으로 통째 교체."""

    changes: list[TempoChangeInput] = Field(default_factory=list, max_length=TEMPO_CHANGES_MAX)

    @model_validator(mode="after")
    def no_duplicate_ticks(self) -> "TempoChangesSaveRequest":
        """같은 위치에 변경점이 둘이면 거부한다 (DB UNIQUE(project_id, tick) 와 같은 규칙)."""
        ticks = [change.tick for change in self.changes]
        if len(ticks) != len(set(ticks)):
            raise ValueError("tempo changes must have distinct ticks")
        return self
```

```
class TempoChangesResponse(BaseModel):
    """템포 변경점 목록 — tick 오름차순."""

    items: list[TempoChangeItem]
    total: int
```

4. 입력 검증

DB의 `CHECK` 제약(`chk_tempo_bpm`, `chk_tempo_tick`)과 `UNIQUE(project_id, tick)`를 요청 단계에서 먼저 같은 규칙으로 막아, DB 오류(500)가 아니라 422로 알려 줍니다.

규칙	값	어기면
`tick`	1 이상 4294967295 이하	422 (0은 시작 템포의 자리)
`bpm`	20 이상 300 이하, 소수 둘째 자리로 반올림	422 (범위 밖), 반올림은 `round_bpm`이 함
`ramp`	`STEP` 또는 `LINEAR`, 생략하면 `STEP`	422
같은 `tick` 두 건	허용하지 않음	422 (`no_duplicate_ticks`)
목록 길이	500건 이하	422 (실수로 큰 목록을 보내는 것을 막음)

5. 왜 이렇게

선택	이유
응답의 `bpm`을 숫자(`float`)로 보냅니다	오디오 엔진(`timing.js`)이 `typeof c.bpm === 'number'`만 받고 문자열이면 오류를 던집니다. 프로젝트 설정의 `bpm`은 `Decimal`이라 문자열로 나가는데, 그 값을 그대로 쓰면 템포 지도 전체가 거부됩니다.
요청도 `float`으로 받고 둘째 자리로 반올림합니다	화면의 점을 끌면 120.456 같은 값이 나옵니다. DB 칸(`DECIMAL(6,2)`)에 맞춰 서버가 반올림하면 저장된 값과 응답이 어긋나지 않습니다.
요청에 `ramp`를 문자열 `Literal`로 둡니다	DB는 `ENUM('STEP','LINEAR')`이고 모델은 `StrEnum`입니다. 요청은 문자열만 받고 서비스에서 `TempoRamp(...)`로 바꿔서, 알 수 없는 값은 모델에 닿기 전에 422로 막힙니다.
시작 템포(tick 0)를 이 표에 넣지 않습니다	DB 설계(DB.md)가 정한 규칙입니다. 시작 템포는 `projects.bpm` 하나가 맡고, 이 표는 변경점만 가져서 두 곳이 어긋날 일이 없습니다.
모델에 `relationship`을 두지 않았습니다	프로젝트 삭제는 DB의 `ON DELETE CASCADE`가 처리하고, 읽기와 쓰기는 `project_id`로 직접 조회합니다. `Project` 모델을 건드리지 않아도 되고 순환 import도 없습니다.

6. 시험

`studionx/backend/tests/test_tempo_routes.py`의 요청 검증 시험이 확인합니다.

- `tick`이 0, 음수, 문자열이면 422, `bpm`이 19.99나 300.01이면 422, `bpm`이나 `tick`이 없으면 422, 알 수 없는 `ramp`면 422입니다.
- 같은 `tick` 두 건이면 422, 501건이면 422입니다.
- 경계값(`tick` 1, `bpm` 20과 300)은 받아들입니다.
- `bpm` 120.456은 120.46으로 저장되고, 응답의 `bpm`은 모두 숫자입니다.

백엔드 309 — 템포 서비스 (`services/tempo_service.py`)

항목	내용
파일	`studionx/backend/app/services/tempo_service.py`
작업	S1-13 템포 변경: 템포 변경점 읽기와 통째 교체, 권한 확인 (PRD 02 FR-202a)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 308 모델과 스키마, 다음 310 API와 `/full`

1. 하는 일

함수	하는 일	권한
`tempo_items`	프로젝트의 템포 변경점을 tick 오름차순으로 읽습니다. 권한 확인은 부르는 쪽이 합니다.	없음
`list_tempo_changes`	멤버인지 확인한 뒤 목록을 돌려줍니다.	멤버면 누구나 (VIEWER 포함)
`replace_tempo_changes`	지금 변경점을 모두 지우고 보낸 목록으로 바꿉니다.	OWNER, EDITOR만

```
replace_tempo_changes(project_id, 보낸 목록)
  멤버 확인 (아니면 404) ─▶ OWNER 또는 EDITOR 확인 (아니면 403)
  한 트랜잭션 안에서: 이 프로젝트의 변경점 전부 삭제 ─▶ 보낸 목록 삽입 ─▶ 커밋
  DB에서 새로 읽어 tick 오름차순으로 돌려줌 (빈 목록을 보내면 모두 지워 평평한 템포로 돌아감)
```

2. 언제 쓰나

쓰는 곳	하는 일
`GET /projects/{id}/tempo-changes` (310)	`list_tempo_changes`
`PUT /projects/{id}/tempo-changes` (310)	`replace_tempo_changes`
`GET /projects/{id}/full` (310)	권한을 이미 확인했으므로 `tempo_items`로 변경점만 읽습니다.

3. 코드

```
def _item(row: TempoChange) -> TempoChangeItem:
    """TempoChange → TempoChangeItem. bpm 은 DECIMAL 을 숫자(float)로 바꿔 보낸다."""
    return TempoChangeItem(
        tempo_change_id=row.tempo_change_id,
        tick=row.tick,
        bpm=float(row.bpm),
        ramp=row.ramp.value,
    )
```

```
def tempo_items(db: Session, project_id: int) -> list[TempoChangeItem]:
    """프로젝트의 템포 변경점을 tick 오름차순으로 읽는다 (권한 확인은 부르는 쪽이 한다).
```

```
    (구현: AI — 2026-10-21, S1-13) 통째 교체 직후에는 세션에 남은 옛 목록과 섞이지 않게
    관계 대신 쿼리로 DB 에서 새로 읽는다.
    """
    rows = db.scalars(
        select(TempoChange).where(TempoChange.project_id == project_id).order_by(TempoChange.tick)
    ).all()
    return [_item(row) for row in rows]
```

```
def list_tempo_changes(db: Session, user: User, project_id: int) -> TempoChangesResponse:
    """프로젝트의 템포 변경점 목록 (FR-202a). 멤버면 누구나."""
    project_service.get_membership(db, project_id, user)
    items = tempo_items(db, project_id)
    return TempoChangesResponse(items=items, total=len(items))
```

```
def replace_tempo_changes(
    db: Session, user: User, project_id: int, data: TempoChangesSaveRequest
) -> TempoChangesResponse:
    """템포 변경점을 보낸 목록으로 통째 교체한다 (FR-202a). OWNER·EDITOR 만.

    (구현: AI — 2026-10-21, S1-13) 지금 붙은 변경점을 모두 지우고 보낸 것을 새로 넣는다.
    한 트랜잭션이라 반쪽 상태가 남지 않는다. 빈 목록이면 모두 지워 처음의 평평한 템포로 돌아간다.
    같은 위치(tick) 중복은 요청 검증(422)이 먼저 막고, 삭제와 삽입을 같은 트랜잭션에서 하므로
    UNIQUE(project_id, tick) 에 걸리지 않는다.
    """
    project, member = project_service.get_membership(db, project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)

    db.execute(delete(TempoChange).where(TempoChange.project_id == project.project_id))
    db.add_all(
        TempoChange(
            project_id=project.project_id,
            tick=change.tick,
            bpm=Decimal(str(change.bpm)),
            ramp=TempoRamp(change.ramp),
        )
        for change in data.changes
    )
    db.commit()
    items = tempo_items(db, project.project_id)
    return TempoChangesResponse(items=items, total=len(items))
```

4. 왜 이렇게

선택	이유
점 하나씩 고치지 않고 목록을 통째로 교체합니다	템포 줄(Tempo lane)은 점을 끌고, 더하고, 지우는 편집이라 곡선 전체가 하나의 상태입니다. 통째로 보내면 서버가 "무엇이 바뀌었는지"를 계산할 필요가 없고, 요청이 중간에 실패해도 반쪽짜리 곡선이 남지 않습니다. 노트를 통째로 바꾸는 `PUT /patterns/{id}/notes`와 같은 방식입니다.
삭제와 삽입을 한 트랜잭션에서 합니다	같은 `tick`에 새 점을 넣을 때 `UNIQUE(project_id, tick)`에 걸리지 않습니다. 같은 요청 안의 중복 `tick`은 스키마가 먼저 422로 막습니다.
교체한 뒤 DB에서 새로 읽어 돌려줍니다	세션에 남은 옛 객체와 새로 넣은 행이 섞이지 않게 하려는 것입니다 (`replace_notes`와 같은 이유). 응답이 곧 저장된 값이라 화면이 그대로 믿고 쓸 수 있습니다.

선택	이유
읽기 함수를 둘로 나눴습니다	`GET /full`은 이미 멤버 확인을 마쳤으므로, 권한 확인이 들어 있는 함수를 다시 부르면 같은 조회가 두 번 일어납니다. 권한 없는 `tempo_items`와 권한 있는 `list_tempo_changes`로 나눠서 쿼리를 한 번만 씁니다.
남의 프로젝트는 404입니다	있는지조차 알려 주지 않는 기존 규칙(`get_membership`)을 그대로 따릅니다. VIEWER는 읽을 수 있지만 저장하면 403입니다.
`bpm`은 `Decimal(str(값))`로 저장합니다	`float`을 그대로 `Decimal`로 바꾸면 0.1 같은 값에 긴 꼬리가 붙습니다. 문자열을 거치면 스키마가 반올림한 값 그대로 들어갑니다.

5. 시험

`studionx/backend/tests/test_tempo_routes.py`가 서비스를 라우터를 통해 확인합니다.

- 비어 있으면 `{items: [], total: 0}`, 저장하면 tick 오름차순으로 돌아옵니다. ramp를 생략하면 `STEP`입니다.
- 교체하면 옛 변경점이 사라지고, 빈 목록을 보내면 모두 지워집니다.
- VIEWER는 읽을 수 있고 저장하면 403이며 값은 그대로입니다. EDITOR는 저장할 수 있습니다.
- 멤버가 아니거나 없는 프로젝트는 읽기와 저장 모두 404이고, 로그인하지 않으면 401입니다.

백엔드 310 — 템포 API와 `/full` (`api/v1/projects.py`)

항목	내용
파일	`studionx/backend/app/api/v1/projects.py`, `studionx/backend/app/services/full_project_service.py`, `studionx/backend/app/schemas/full_project.py`
작업	S1-13 템포 변경: 템포 변경점 조회와 저장 API, 그리고 프로젝트 전체 조회에 변경점 포함 (PRD 02 FR-202a, FR-254)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 309 템포 서비스, 다음 311 화면과 오디오 연결

1. 하는 일

요청	하는 일	권한
`GET /projects/{id}/tempo-changes`	템포 변경점 목록 (tick 오름차순)	멤버면 누구나
`PUT /projects/{id}/tempo-changes`	보낸 목록으로 통째 교체. 빈 목록이면 모두 지움	OWNER, EDITOR (VIEWER는 403)
`GET /projects/{id}/full`	응답의 `tempo_changes`가 이전에는 늘 빈 목록이었고, 이제 저장된 변경점을 담음	멤버면 누구나

라우터는 얇게 두고 로직은 `tempo_service`(309)에 있습니다. 요청과 응답의 모양은 308의 스키마입니다.

```
PUT /api/v1/projects/12/tempo-changes
{ "changes": [ { "tick": 3840, "bpm": 140, "ramp": "STEP" },
               { "tick": 7680, "bpm": 160.5, "ramp": "LINEAR" } ] }

200 { "items": [ { "tempo_change_id": 31, "tick": 3840, "bpm": 140.0, "ramp": "STEP" },
                 { "tempo_change_id": 32, "tick": 7680, "bpm": 160.5, "ramp": "LINEAR" } ],
      "total": 2 }
```

2. 언제 쓰나

쓰는 곳	하는 일
시퀀서가 화면을 열 때	`GET /full`의 `tempo_changes`를 받아 재생과 내보내기가 씁니다 (311).
템포 줄(Tempo lane) 편집 (아직 화면이 없음)	점을 고칠 때마다 `PUT`으로 전체 목록을 보냅니다.

3. 코드

3.1 라우터

```
@router.get(
    "/{project_id}/tempo-changes",
    response_model=TempoChangesResponse,
    responses=NOT_FOUND,
)
```

```
def read_tempo_changes(project_id: int, user: CurrentUser, db: Db) -> TempoChangesResponse:
    """프로젝트의 템포 변경점 목록 (FR-202a) — 멤버면 누구나, tick 오름차순."""
    return tempo_service.list_tempo_changes(db, user, project_id)
```

```
@router.put(
    "/{project_id}/tempo-changes",
    response_model=TempoChangesResponse,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def replace_tempo_changes(
    project_id: int,
    data: TempoChangesSaveRequest,
    user: CurrentUser,
    db: Db,
) -> TempoChangesResponse:
    """템포 변경점을 보낸 목록으로 통째 교체 (FR-202a) — OWNER·EDITOR 만 (VIEWER 는 403).

    템포 줄(Tempo lane)이 바뀔 때마다 점 전체를 보낸다. 빈 목록이면 모두 지워 평평한 템포로 돌아감.
    """
    return tempo_service.replace_tempo_changes(db, user, project_id, data)
```

3.2 프로젝트 전체 조회에 변경점 포함

`get_full_project`의 끝에서 `FullProjectResponse`를 만들 때, 이전에는 `tempo_changes`를 채우지 않아 기본값인 빈 목록이 나갔습니다. 이제 `tempo_service.tempo_items`로 읽어 넣습니다.

```
    return FullProjectResponse(
        project=project_info(db, project, member),
        tracks=[...],
        patterns=[...],
        clips=clips,
        tempo_changes=tempo_service.tempo_items(db, project_id),
    )
```

응답 스키마에서는 `tempo_changes`의 타입이 `list[dict[str, Any]]`에서 `list[TempoChangeItem]`으로 바뀌었습니다. 화면이 읽는 모양은 같은 JSON이고, 이제 모양이 정해져 있다는 점만 다릅니다 (`tempo_change_id`, `tick`, `bpm`, `ramp`).

```
class FullProjectResponse(BaseModel):
    """GET /projects/{id}/full 응답 — 한 번의 요청으로 곡 전체 (FR-254).

    clips 는 프로젝트의 모든 클립(track_id 로 트랙에 묶임), connections 는 각 트랙의
    나가는 연결이다. tempo_changes 는 저장된 템포 변경점(tick 오름차순)이고, markers ·
    meter_changes 는 그 표와 API 가 아직 없어 늘 빈 목록이다.
    """

    project: FullProjectInfo
    tracks: list[FullTrackResponse]
    patterns: list[PatternResponse]
    clips: list[ClipResponse] = Field(default_factory=list)
    markers: list[dict[str, Any]] = Field(default_factory=list)
    tempo_changes: list[TempoChangeItem] = Field(default_factory=list)
    meter_changes: list[dict[str, Any]] = Field(default_factory=list)
```

4. 왜 이렇게

선택	이유
새 라우터 파일 대신 `projects.py`에 둡니다	변경점은 프로젝트에 속하고 주소가 `/projects/{id}/...`입니다. `/full`과 `/settings`가 이미 같은 파일에 있어서 곡 전체 설정과 한곳에서 보입니다.
`PUT`(통째 교체)이고 `POST`와 `PATCH`와 `DELETE`가 없습니다	템포 줄은 점 전체가 하나의 곡선이라, 점마다 주소를 두면 요청이 많아지고 중간 상태가 생깁니다 (309).
`GET /full`에도 넣어 화면이 한 번에 받습니다	시퀀서는 화면을 열 때 서버 요청이 한 번이어야 한다는 S1-10의 원칙을 지킵니다. 별도 `GET`은 템포 줄만 새로 읽을 때 쓰도록 남겨 둡니다.
`/full`의 `tempo_changes` 타입을 구체적으로 바꿨습니다	`dict[str, Any]`는 "아직 모델이 없어 빈 목록"이라는 임시 자리였습니다. 모델이 생겼으니 모양을 고정해야 OpenAPI 문서와 화면이 같은 약속을 봅니다.
`markers`와 `meter_changes`는 그대로 빈 목록입니다	그 표의 모델과 API는 이번 범위가 아닙니다. 화면은 이 칸이 늘 있다고 보고 읽으므로 칸 자체는 유지합니다.

5. 시험

`studionx/backend/tests/test_tempo_routes.py`(22건)와 `test_full_project_routes.py`가 확인합니다.

- 연결: `/projects/{project_id}/tempo-changes`에 `get`과 `put`만 등록되어 있습니다.
- `GET /full`의 `tempo_changes`가 저장한 변경점을 tick 오름차순으로 담고, 각 항목의 키가 `tempo_change_id`, `tick`, `bpm`, `ramp`입니다.
- 변경점이 없는 프로젝트는 `tempo_changes`가 빈 목록입니다 (기존 시험이 그대로 통과).
- 백엔드 전체 시험 1105건이 통과합니다 (이전 1083건에 22건 추가). `ruff`도 통과합니다.

JavaScript 311 — 템포 변경점을 재생, 재생선, 내보내기에 연결 (`sequencer-devices.js`, `sequencer-audio.js`, `sequencer-arrange.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`, `studionx/frontend/static/js/pages/sequencer-audio.js`, `studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-13 템포 변경: `GET /full`의 템포 변경점을 담아 두고, 재생 세션, 재생선과 시간 표시, WAV 내보내기가 같은 목록으로 시간을 계산하게 함 (FR-202a)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 310 템포 API와 `/full`, 다음 없음 (S1-13 템포 변경의 마지막 문서)

1. 하는 일

오디오 쪽(`timing.js`, `scheduler.js`, `offline.js`)은 템포 변경을 처음부터 지원했습니다. `ticksToSecondsWithChanges`가 변경점 목록으로 시간을 계산하고, 내보내기의 길이와 범위 계산도 그 목록을 받습니다. 그런데 화면이 그 목록을 한 번도 넘기지 않았습니다. 재생 세션은 `getTempoChanges`를 받지 못했고, 내보내기 스냅샷은 `tempoChanges: []`를 고정으로 넘겼으며, 화면의 재생선과 시간 표시는 시작 템포 하나로 계산했습니다.

```
서버  GET /projects/{id}/full ─▶ tempo_changes: [ { tempo_change_id, tick, bpm, ramp } ]
화면  applyFullProject ─▶ indexTempoChanges: { tick, bpm, ramp } 로 정리해 담아 둠
        ├ playbackTempoChanges()를 PlaybackSession 의 getTempoChanges 로 넘김 ─▶ 재생 위치와 예약 시각이 템포 지도를 따름
        ├ exportSnapshot().tempoChanges 로 넘김 ─▶ 내보내기의 길이, 범위, 렌더링이 템포 지도를 따름
        └ setTempoChangesSource(playbackTempoChanges) ─▶ 재생선 이동과 상단 시간 표시가 템포 지도를 따름
```

고치기 전에는 템포 변경이 있는 곡이라도 재생과 내보내기, 재생선과 시간 표시가 모두 시작 템포 하나로 계산했습니다. 둘째 마디부터 템포가 절반인 곡은 내보낸 파일이 실제보다 짧았고 (3.87초, 올바른 길이는 5.81초), 재생선은 소리보다 앞서 달렸으며 시간 표시도 실제 경과 시간보다 작았습니다.

2. 언제 쓰나

쓰는 곳	하는 일
`applyFullProject` (273)	프로젝트를 열 때 `indexTempoChanges`로 변경점을 담습니다.
`getSession` (296)	재생 세션을 만들 때 `getTempoChanges: playbackTempoChanges`를 넘깁니다.
`exportSnapshot` (296)	내보내기 창이 쓰는 스냅샷의 `tempoChanges`에 `playbackTempoChanges()`를 넣습니다.
`initAudio` (296)	`setTempoChangesSource(playbackTempoChanges)`로 재생선과 시간 표시가 같은 목록을 읽게 합니다.
`play`의 재생선 이동, `updateTime` (`sequencer-arrange.js`)	변경점이 있으면 템포 지도로 위치와 시간을 계산합니다.

3. 코드

3.1 변경점 담기와 꺼내기 (`sequencer-devices.js`)

```
function indexTempoChanges(list) {
  tempoChanges = (list || []).map(function (c) {
    return { tick: Number(c.tick), bpm: Number(c.bpm), ramp: c.ramp === 'LINEAR' ? 'LINEAR' : 'STEP' };
  }).filter(function (c) {
    return isFinite(c.tick) && c.tick > 0 && isFinite(c.bpm) && c.bpm > 0;
  }).sort(function (a, b) { return a.tick - b.tick; });
}
```

```
export function playbackTempoChanges() {
  return tempoChanges.map(function (c) { return { tick: c.tick, bpm: c.bpm, ramp: c.ramp }; });
}
```

3.2 오디오와 연결 (`sequencer-audio.js`)

바뀐 곳은 세 군데이고, 모두 한 줄입니다. 위의 함수를 `import`에 더하고 아래처럼 넘깁니다.

```
// getSession 안: 재생 세션이 템포 지도를 받음 (이전에는 이 줄이 없었음)
session = new PlaybackSession({
  getTracks: playbackTracks,
  ...
  getBpm: currentBpm,
  getTempoChanges: playbackTempoChanges,
  ...
});

// exportSnapshot 안: 이전에는 tempoChanges: [] 로 고정
tempoChanges: playbackTempoChanges(),

// initAudio 안: 재생선과 시간 표시가 같은 템포 지도를 읽음
setTempoChangesSource(playbackTempoChanges);
```

3.3 재생선과 시간 표시 (`sequencer-arrange.js`)

재생선은 소리 엔진과 따로 움직입니다. 화면이 `requestAnimationFrame`마다 지난 시간만큼 위치를 옮기는데, 이전에는 "시작 템포에서 1초에 몇 마디"를 곱했습니다. 템포 변경점이 있으면 지금 위치의 시각(초)에 지난 시간을 더한 뒤 그 시각의 위치(tick)로 바꿉니다. 시간 표시도 같은 변환을 씁니다.

```
export function setTempoChangesSource(fn) {
  tempoSource = typeof fn === 'function' ? fn : null;
}
```

```
function currentTempoChanges() {
  return tempoSource ? tempoSource() : [];
}
```

```
function play() {
  const c = playheadCtx;
  c.playing = true;
```

```
  c.startBar = c.bar;
  c.anchor = null;
  if (transportHooks.onPlay) transportHooks.onPlay(c.bar);
  document.body.classList.add('is-playing');
  let last = performance.now();
  const tick = (now) => {
    if (!c.playing) return;
    const dt = (now - last) / 1000;
    const changes = currentTempoChanges();
    let next;
    if (changes.length) {
      // 템포 변경점이 있으면 지금 위치의 시각에 지난 시간을 더한 뒤 다시 위치(tick)로 바꿈 (소리와 같은 템
포 지도)
      const sec = ticksToSecondsWithChanges(c.bar * TICKS_PER_BAR, currentBpm(), changes);
      next = secondsToTicks(sec + dt, currentBpm(), changes) / TICKS_PER_BAR;
    } else {
      next = c.bar + dt * (currentBpm() / 60 / 4);     // 4/4, 템포가 하나일 때
    }
    // Loop 켜짐: 구간 끝을 지나면 구간 처음으로 (구간보다 뒤에서 시작했으면 그대로 진행)
    const loopEnd = loopState.start + loopState.length;
    if (loopState.on && loopState.length > 0 && c.bar < loopEnd && next >= loopEnd) next = loopState.sta
rt + ((next - loopEnd) % loopState.length);
    last = now;
    if (next >= totalBars(c.lanes)) { stop(); return; }
    setPlayheadBar(next);
    c.raf = requestAnimationFrame(tick);
  };
  c.raf = requestAnimationFrame(tick);
}
```

```
function updateTime(bar) {
  const main = document.getElementById('time-main');
  const small = document.getElementById('time-bars');
  if (!main || !small) return;
  const changes = currentTempoChanges();
  const sec = changes.length
    ? ticksToSecondsWithChanges(bar * TICKS_PER_BAR, currentBpm(), changes)
    : (bar * 4 * 60) / currentBpm();
  const mm = String(Math.floor(sec / 60)).padStart(2, '0');
  const ss = String(Math.floor(sec % 60)).padStart(2, '0');
  const ms = String(Math.floor((sec * 1000) % 1000)).padStart(3, '0');
  const clock = `${mm}:${ss}:${ms}`;
  const bars = `${Math.floor(bar) + 1}.${Math.floor((bar % 1) * 4) + 1}.${Math.floor(((bar * 4) % 1) *
4) + 1}`;
  const barsFirst = document.getElementById('time')?.dataset.mode === 'bars';
  main.textContent = barsFirst ? bars : clock;
  small.textContent = barsFirst ? clock : bars;
}
```

4. 최신 문법으로 쓰면

```
// indexTempoChanges 를 최신 문법으로 쓴 모양
const indexTempoChanges = (list = []) => {
  tempoChanges = list
```

```
    .map(({ tick, bpm, ramp }) => ({ tick: Number(tick), bpm: Number(bpm), ramp: ramp === 'LINEAR' ? 'LI
NEAR' : 'STEP' }))
    .filter(({ tick, bpm }) => Number.isFinite(tick) && tick > 0 && Number.isFinite(bpm) && bpm > 0)
    .sort((a, b) => a.tick - b.tick);
};

export const playbackTempoChanges = () => tempoChanges.map((c) => ({ ...c }));
```

전통 문법 (실제 코드)	최신 문법
`(list || [])`	기본 매개변수 `list = []`
`function (c) { return { ... }; }`	화살표 함수
`{ tick: c.tick, bpm: c.bpm, ramp: c.ramp }`로 복사	객체 전개 `{ ...c }`
`isFinite(x)`	`Number.isFinite(x)`

5. 왜 이렇게

선택	이유
목록을 `sequencer-devices.js`에 둡니다	클립, 패턴, 트랙처럼 `GET /full`로 받은 프로젝트 데이터를 이 파일이 이미 담고 있고, 재생과 내보내기가 `playbackTracks`, `playbackInstruments`와 같은 방식으로 읽어 갑니다. 새 저장소를 만들 필요가 없습니다.
재생과 내보내기가 같은 함수를 부릅니다	들리는 소리와 파일의 소리가 같아야 한다는 원칙(S1-11b)을 템포에도 적용합니다. 한쪽만 고치면 두 결과가 어긋납니다.
`playbackTempoChanges`는 복사본을 돌려줍니다	엔진이 목록을 정렬하거나 고쳐도 화면이 가진 원본이 변하지 않게 합니다.
숫자가 아니거나 `tick`이 0 이하인 줄은 버립니다	`timing.js`는 이런 줄을 만나면 오류를 던져 재생이 멈춥니다. 서버가 이미 검증하지만, 화면 쪽에서도 한 번 거르면 이상한 데이터 하나가 재생 전체를 막지 않습니다.
`ramp`는 `LINEAR`만 인정하고 나머지는 `STEP`입니다	엔진의 규칙과 같습니다 (`timing.js`가 `LINEAR`가 아니면 `STEP`으로 취급).
재생선은 위치를 시각으로 바꿨다가 다시 위치로 바꿉니다	템포가 바뀌는 곳을 지나는 프레임에서도 정확하게 이어지게 하려는 것입니다. 현재 템포를 곱하는 방식은 변경점을 지나는 프레임에서 오차가 생깁니다.
변경점이 없으면 이전 계산 그대로입니다	템포가 하나뿐인 곡(대부분의 곡)은 바뀐 것이 없고, 기존 시험이 모두 그대로 통과합니다.
재생선 등록은 함수 하나로 합니다 (`setTempoChangesSource`)	`sequencer-arrange.js`가 `sequencer-devices.js`를 `import`하면 서로를 부르는 순환이 생깁니다 (devices가 이미 arrange를 `import`). 이미 있는 `setTransportHooks`와 같은 방식으로 오디오 연결 파일이 목록을 알려 주게 했습니다.

6. 알려진 한계

한계	내용
템포를 화면에서 편집하는 방법이 아직 없습니다	템포 줄(Tempo lane)은 `sequencer.js`의 `renderGlobalLanes`로 정해져 있지만 아직 구현되지 않은 템플릿입니다. 지금은 API(`PUT /projects/{id}/tempo-changes`)로만 변경점을 만들 수 있습니다.
박자 변경은 아직 반영하지 않습니다	재생선과 시간 표시는 4/4로 계산합니다. 박자 변경점(`meter_changes`)은 모델과 API가 아직 없습니다.

한계	내용
마디 칸의 눈금은 그대로입니다	마디선 간격은 마디 단위라 템포와 무관해서 바뀌지 않습니다. 템포가 바뀌면 같은 마디가 더 오래 또는 짧게 걸릴 뿐입니다.
딜레이의 템포 동기(Sync)는 시작 템포를 씁니다	이펙트가 받는 `getBpm`은 곡 전체에서 하나의 값입니다. 곡 중간에서 템포가 바뀌어도 Sync 딜레이의 간격은 바뀌지 않습니다.

7. 시험

`docs/handoff/s1-13_tests/ui_tempo.mjs`(20건)가 실제 화면(헤드리스 Chrome)에서 확인합니다. 시험 전용 오디오 트랙에 8초 사인파 클립(2마디)을 놓고, 둘째 마디부터 템포를 절반으로 낮춥니다.

- 서버: `GET /full`의 `tempo_changes`에 변경점 한 건이 있고 `bpm`이 숫자입니다.
- 화면: `playbackTempoChanges()`와 `exportSnapshot().tempoChanges`가 같은 목록(tick 3840, 절반 템포, STEP)이고, 상단 BPM은 프로젝트의 시작 템포 그대로입니다.
- 재생: 엔진의 `transport.changes`가 변경점을 받았고, 재생 위치가 템포 지도로 계산한 위치와 200 tick 안으로 맞으며 시작 템포 그대로라면 있을 위치보다 500 tick 넘게 뒤에 있습니다.
- 재생선과 시간 표시: 재생선이 엔진 위치와 0.2마디 안으로 맞고(화면은 곧바로 시작하고 엔진은 샘플을 불러온 뒤 시작하므로 그 차이를 허용), 시작 템포로 계산했을 위치보다 분명히 뒤에 있습니다. 상단 시간 표시가 소리의 경과 시간과 0.6초 안으로 맞고, 시작 템포로 계산한 시간과는 0.5초 넘게 다릅니다.
- 내보내기 창: 보여 주는 전체 길이가 템포 지도 기준(5.8초 근처)이고, 변경을 무시한 길이(3.9초)와 다릅니다. 실제로 내려받은 WAV의 길이가 5.81초이고, 느려진 둘째 마디의 끝까지 소리가 이어집니다.
- 되돌려 보기: `sequencer-audio.js`의 연결만 원래대로 돌려놓고 시험을 돌리면(재생선과 시간 표시를 넣기 전의 16건 시험) 6건이 실패합니다 (엔진이 변경점을 못 받고, 파일 길이 3.87초). 즉 이 시험이 이 오류를 실제로 잡습니다.
- 시험 중 발견한 문제는 없었습니다. 위 되돌려 보기가 이 작업에서 고친 오류(재생과 내보내기가 템포 변경을 무시)의 증거입니다.